

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ № 2

"Доступность в HTML "

Выполнил:
Демьянов Фёдор

Группа: J3214

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Адаптировать пользовательский интерфейс проекта NovaTransit под стандарты доступности WAI-ARIA и рекомендации WCAG 2.1 (уровень AA). Сделать так, чтобы сайтом было одинаково удобно пользоваться как с обычной клавиатуры без мыши, так и с помощью программ экранного доступа.

2. Краткие теоретические сведения

Доступность веб-интерфейсов строится вокруг базовых принципов стандарта WCAG 2.1, объединенных аббревиатурой POUR:

- **Восприимчивость (Perceivable):** весь контент должен быть понятен органам чувств. Если на странице есть графика, у нее должны быть текстовые описания, а цвета не должны сливаться с фоном.
- **Управляемость (Operable):** сайтом можно полноценно управлять без мыши, только клавишами Tab, Enter, Space и стрелками. При этом фокус не должен теряться или застревать в модальных окнах.
- **Понятность (Understandable):** интерфейс ведет себя предсказуемо, а подсказки и ошибки форм четко объясняют, что пошло не так.
- **Надежность (Robust):** код корректно читается разными браузерами и вспомогательными программами вроде NVDA или VoiceOver.

Для тех случаев, где стандартных тегов HTML не хватает для объяснения сложного динамического интерфейса, используется спецификация WAI-ARIA. Она дает возможность явно указать роль элемента, его состояние, контекстное имя и логические связи с другими узлами страницы.

3. Что конкретно было сделано в проекте

Я последовательно доработал все пять страниц сайта ([index.html](#), [tour.html](#), [dashboard.html](#), [login.html](#), [register.html](#)), таблицу стилей [style.css](#) и скрипты, которые отвечают за отрисовку динамических данных ([catalog.js](#), [tour.js](#), [dashboard.js](#), [nasa.js](#), [auth.js](#)). При этом весь визуальный дизайн и поведение клиент-серверной части остались прежними.

3.1. Ориентиры и структура страниц

- На главной странице [index.html](#) я вынес основной контент из прямого подчинения `<body>` и обернул промо-блок с каталогом в семантический тег `<main id="main-content">`. Теперь скринридер видит четкую точку входа в контент.
- В начало всех страниц я добавил скрытую ссылку быстрого перехода. Зрячие пользователи ее не видят, но если нажать Tab сразу после загрузки страницы, появляется кнопка «Перейти к основному содержимому», которая позволяет пропустить шапку и не перебирать все пункты меню каждый раз.
- Выстроил строгую иерархию заголовков. Раньше на страницах входа и регистрации были только `<h2>`, а в личном кабинете заголовка первого уровня не было вовсе. Я

исправил это: теперь на каждой странице есть ровно один `<h1>`. Промежуточные подзаголовки внутри карточек и виджетов приведены к уровням `<h2>` и `<h3>`, поэтому скринридер больше не спотыкается на резких перескоках к `<h4>` или `<h6>`.

- Карточки экспедиций и отзывы кадетов перевел на теги `<article>`, а смысловые блоки страниц разметил тегами `<section>` с привязкой к заголовкам через `aria-labelledby`.
- Нашел и убрал баг в навбаре, где атрибут `aria-current="page"` по ошибке стоял сразу на двух ссылках одновременно.

3.2. Кнопки, ссылки и текстовые альтернативы

- В шаблоне генерации карточек каталога (`catalog.js`) исправил кнопки «Подробнее». Раньше скринридер зачитывал шесть одинаковых слов подряд. Я добавил атрибут `aria-label`, в который подставляется полное название тура, например, «Подробнее об экспедиции Марсианский транзит».
- На странице тура (`tour.html`) разметил карусель со снимками. Кнопкам переключения слайдов добавил скрытый текст «Предыдущий снимок» и «Следующий снимок», а индикаторам внизу прописал понятные подписи с номерами кадров.
- Кнопке мобильного меню навбара задал недостающие атрибуты `aria-label`, `aria-controls` и `aria-expanded`.
- Скрыл весь декоративный визуальный шум: иконки Bootstrap Icons и фоновые картинки планет теперь помечены атрибутом `aria-hidden="true"`, чтобы синтезатор речи их просто игнорировал.
- Спиннерам загрузки каталога и виджета NASA добавил текстовые пояснения внутри класса `.visually-hidden`, чтобы незрячий пользователь понимал, что прямо сейчас идет запрос к базе.
- В скрипте авторизации (`auth.js`) настроил динамическую подстановку имени кадета в атрибут `alt` аватарки в шапке вместо стандартного слова `Avatar`.

3.3. Доступность форм и валидация

- Всем обязательным полям ввода в формах входа, регистрации и бронирования добавил атрибут `aria-required="true"`.
- Связал сообщения об ошибках валидации с инпутами через `aria-describedby`. Если пользователь введет некорректный email или ошибется в пароле, скринридер прочитает текст всплывшей ошибки под полем.
- Блок выбора дополнительных модулей в модальном окне заказа оформил через семантическую пару `<fieldset>` и `<legend>`.

- Модальным окнам прописал `role="dialog"`, `aria-modal="true"` и связал их с заголовками через `aria-labelledby`.

3.4. Таблицы, прогресс-бары и виджет NASA

- В личном кабинете навел порядок в таблице полетного журнала: добавил скрытый заголовок `<caption>` и проставил атрибуты `scope="col"` для шапки таблицы. Теперь вспомогательные технологии понимают структуру строк и столбцов.
- Шкалы предполетных нормативов связал с текстовыми названиями тестов с помощью `aria-labelledby`. Раньше программа экранного доступа читала просто «80%», а теперь четко произносит название дисциплины и процент ее прохождения.
- В модуле `nasa.js` для встроенного видеоплеера `<iframe>` добавил обязательный атрибут `title` с названием видео дня. Если видео открывается во внешней вкладке, добавил скрытое предупреждение для скринридера, чтобы переход не стал неожиданностью.

3.5. Стили клавиатурного фокуса

- В `style.css` я настроил поведение фокуса через `:focus-visible`. При перемещении по сайту клавишей Tab вокруг ссылок, кнопок и полей появляется серебристая контрастная рамка, по которой легко отслеживать свое положение на странице. При клике мышью эта рамка не показывается и не портит визуал.
- Добавил стандартные CSS-классы `.visually-hidden` и `.visually-hidden-focusable`, которые надежно прячут служебный текст с экрана, но оставляют его в дереве доступности браузера.

4. Результаты проверки

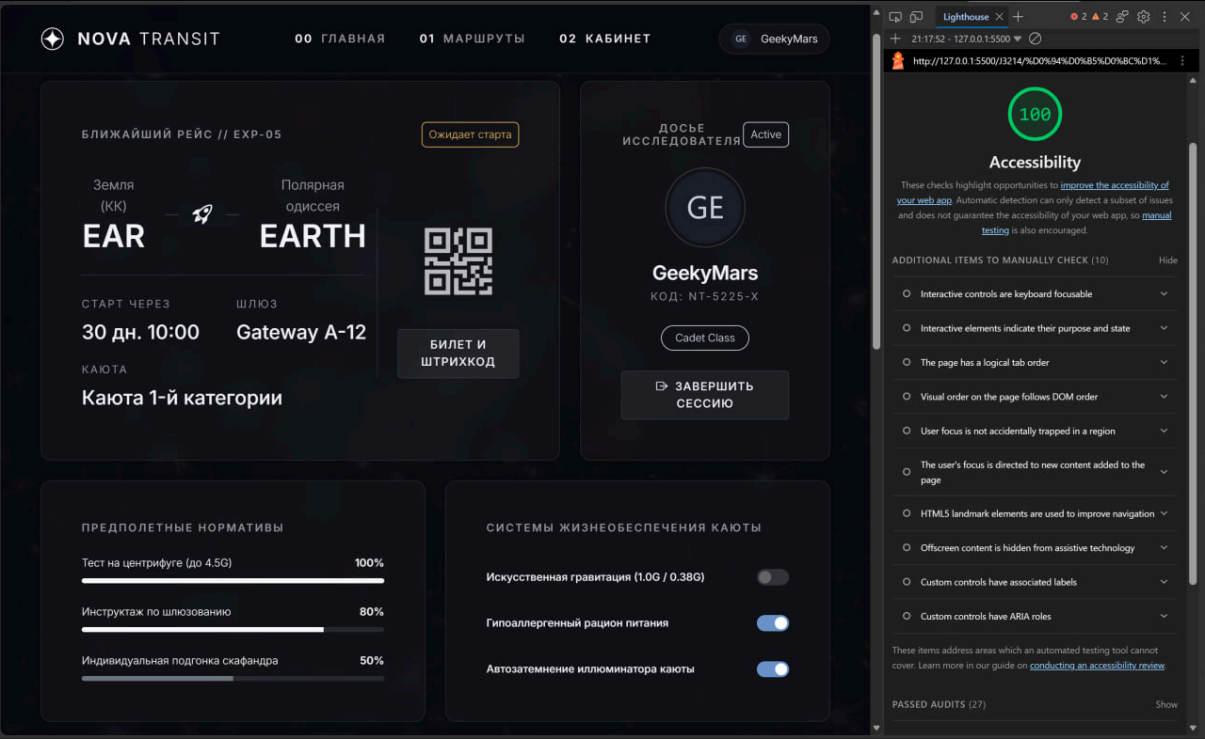
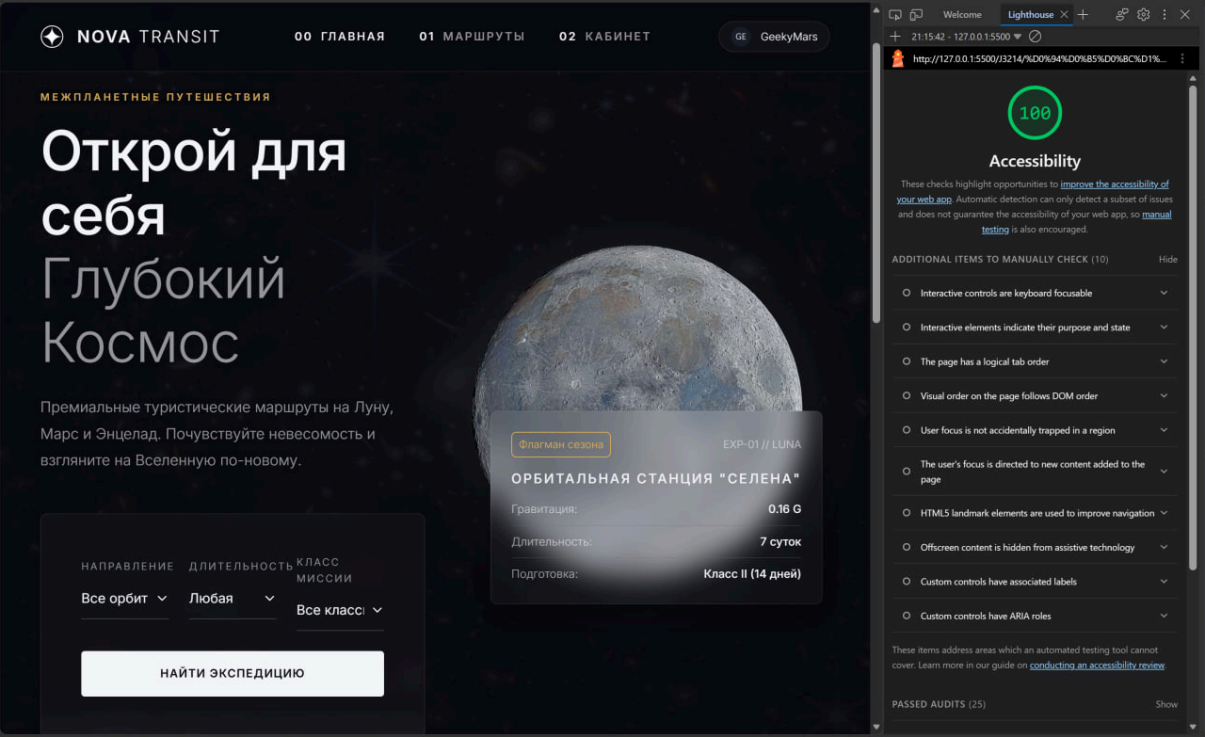
Для контроля доступности я использовал инструмент: Google Lighthouse

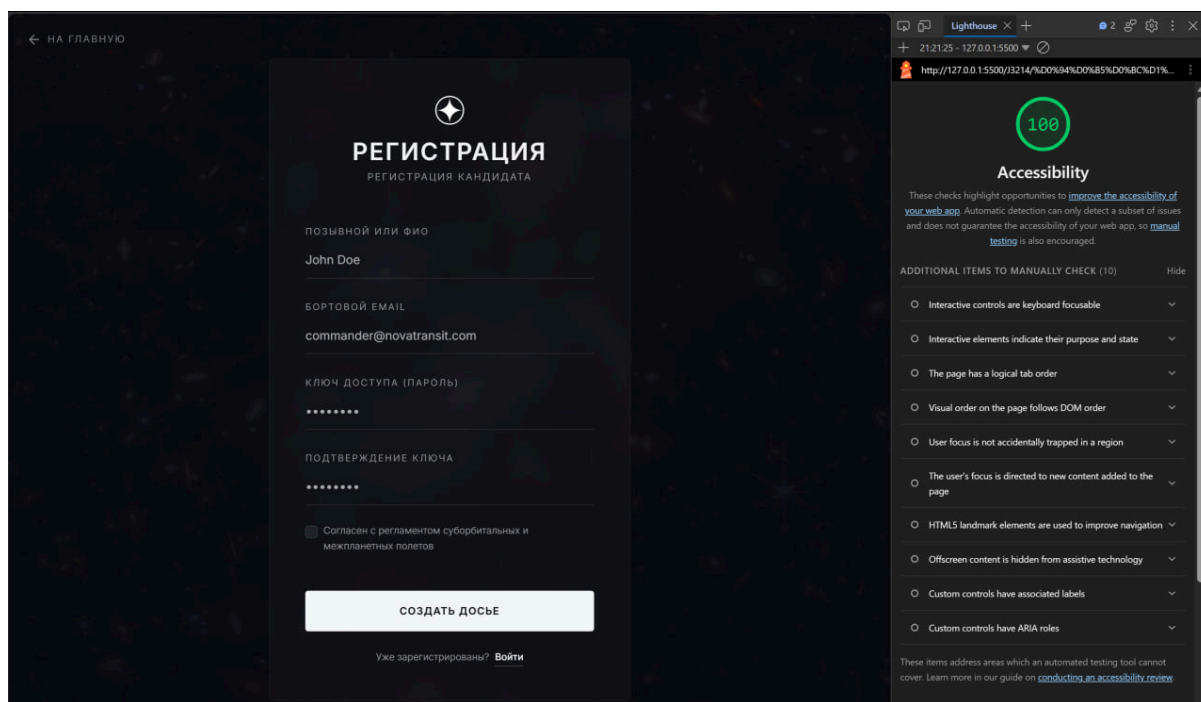
Проверка в Google Lighthouse

Я протестировал все страницы проекта, выбрав категорию Accessibility:

- `index.html` - 100 / 100 баллов.
- `tour.html?id=enceladus` - 100 / 100 баллов.
- `dashboard.html` - 100 / 100 баллов.
- `login.html` - 100 / 100 баллов.
- `register.html` - 100 / 100 баллов.

Все тесты на доступные имена кнопок, контрастность, атрибуты `alt`, заголовки и структуру ориентиров пройдены без единого замечания.





5. Заключение

До выполнения этой работы мне казалось, что верстка заканчивается там, где макет визуально совпадает с задумкой и все кнопки кликабельны и работают. Но когда начинаешь проверять интерфейс инструментами доступности или пытаешься пройти путь пользователя исключительно с клавиатуры с закрытыми глазами, быстро понимаешь реальную картину. Без правильных ориентиров, атрибутов WAI-ARIA и связок ошибок сайт для скринридера превращается в набор безымянных ссылок и непонятных картинок.

Эта работа оказалась очень полезной практикой. Я научился смотреть на DOM-дерево глазами браузера и вспомогательных технологий. Разобраться с `:focus-visible`, связать подсказки валидации через `aria-describedby` и разметить карусели оказалось технически несложно, но это в корне меняет качество продукта для людей с ограничениями по зрению или моторике.